CSE STUDY HUB

C++ Programming

UNIT 5 · Concurrent Programming and Modern Applications

What this unit is about

This unit builds understanding of **Concurrent Programming and Modern Applications**. These notes are designed for quick learning and exam revision: learn the meaning, understand the working, practise an example, and finish with comparisons and key terms.

Syllabus coverage checklist

1. Concurrency vs parallelism; processes and threads

2. Creating and managing threads with std::thread

3. Data races, critical sections and mutex/lock_guard

4. Deadlock conditions and prevention strategies

5. Atomic variables and memory-operation basics

6. Tasks and asynchronous work with std::async

7. Promises, futures and shared_futures

8. C++20 coroutines: co_await, co_yield and co_return; use cases

How to study: Read each topic once for meaning, underline definitions and formulas, reproduce diagrams/steps without looking, then answer the revision questions on Page 5.

Exam answer pattern

For a 5-mark answer: (1) definition, (2) labelled diagram or formula, (3) working/components in steps, (4) example/application, and (5) one advantage, limitation or comparison.

UNIT 5 NOTES · PART 1

Concurrency vs parallelism; processes and threads

Compare the alternatives in Concurrency vs parallelism; processes and threads using the same criteria: purpose, working method, performance, cost/complexity, security or reliability, and suitable use case. End with a short conclusion stating when each option is preferable.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

Creating and managing threads with std::thread

Creating and managing threads with std::thread is a syllabus concept in C++ Programming. Learn its definition, purpose, key terms and working principle. Organise the explanation into components or steps, then add a practical example, one advantage, one limitation and a comparison with a related concept.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

Data races, critical sections and mutex/lock_guard

A mutex protects a critical section so only one thread accesses shared mutable state at a time. Use RAII locks such as lock_guard/unique_lock and acquire multiple locks consistently to reduce deadlock risk.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

Connect and apply

For a practical question on Concurrent Programming and Modern Applications, first identify which syllabus concept is being tested. State assumptions, draw the relevant structure/flow, work through the steps in order, and finish by interpreting the result. Mention complexity, reliability, accuracy, cost or ethical impact whenever it is relevant.

UNIT 5 NOTES · PART 2

Deadlock conditions and prevention strategies

For Deadlock conditions and prevention strategies, identify the given values and the quantity to find before selecting a formula. Keep units consistent, show substitutions, calculate carefully and perform a reasonableness check. Write the formula and one solved example in your revision copy.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

Atomic variables and memory-operation basics

Start with a precise definition of Atomic variables and memory-operation basics. Explain its purpose, the problem it solves, its main components and where it is used. In an exam answer, add one practical example and distinguish it from the nearest related concept.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

Tasks and asynchronous work with std::async

Tasks and asynchronous work with std::async is a syllabus concept in C++ Programming. Learn its definition, purpose, key terms and working principle. Organise the explanation into components or steps, then add a practical example, one advantage, one limitation and a comparison with a related concept.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

Connect and apply

For a practical question on Concurrent Programming and Modern Applications, first identify which syllabus concept is being tested. State assumptions, draw the relevant structure/flow, work through the steps in order, and finish by interpreting the result. Mention complexity, reliability, accuracy, cost or ethical impact whenever it is relevant.

UNIT 5 NOTES · PART 3

Promises, futures and shared_futures

Promises, futures and shared_futures is a syllabus concept in C++ Programming. Learn its definition, purpose, key terms and working principle. Organise the explanation into components or steps, then add a practical example, one advantage, one limitation and a comparison with a related concept.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

C++20 coroutines: co_await, co_yield and co_return; use cases

For C++20 coroutines: co_await, co_yield and co_return; use cases, connect the concept to a practical scenario. Identify the input or need, the technology/process used, the output or benefit, and one limitation. Strong examples explain not only what is used but why it is suitable.

Write in your own words: Define the concept, list its important parts/steps, and attach one example from a system, program, workplace or numerical problem.

Connect and apply

For a practical question on Concurrent Programming and Modern Applications, first identify which syllabus concept is being tested. State assumptions, draw the relevant structure/flow, work through the steps in order, and finish by interpreting the result. Mention complexity, reliability, accuracy, cost or ethical impact whenever it is relevant.

QUICK REVISION & EXAM PRACTICE

Key takeaways

• **Concurrency vs parallelism; processes and threads** — know the definition, core working and one example.

• **Creating and managing threads with std::thread** — know the definition, core working and one example.

• **Data races, critical sections and mutex/lock_guard** — know the definition, core working and one example.

• **Deadlock conditions and prevention strategies** — know the definition, core working and one example.

• **Atomic variables and memory-operation basics** — know the definition, core working and one example.

• **Tasks and asynchronous work with std::async** — know the definition, core working and one example.

• **Promises, futures and shared_futures** — know the definition, core working and one example.

• **C++20 coroutines: co_await, co_yield and co_return; use cases** — know the definition, core working and one example.

Worked-answer framework

Scenario: A question asks you to explain or apply a concept from Concurrent Programming and Modern Applications. **Step 1:** identify the relevant term and define it. **Step 2:** write the formula, components or ordered process. **Step 3:** apply it to the given data/scenario. **Step 4:** state the result and justify it. **Step 5:** mention one limitation or precaution.

Self-test questions

Q1. Define Concurrency vs parallelism; processes and threads and explain its importance.

Q2. Explain the working/steps involved in Creating and managing threads with std::thread.

Q3. Differentiate between two related concepts from Concurrent Programming and Modern Applications.

Q4. Give one practical application of C++20 coroutines: co_await, co_yield and co_return; use cases and discuss one limitation.

Q5. Write a structured 5-mark answer covering the main ideas of Concurrent Programming and Modern Applications.

Final checklist: I can define every topic · explain the process without notes · reproduce formulas/diagrams · solve one example · compare related concepts.